

LEARNING RESOURCE GUIDE

ISC2 CCSP: Certified Cloud Security Professional (Aug 2026) | Professional Participant Workbook

Learning Objectives

By the end of this lecture and its accompanying guide, you will be able to:

- By the end of this lecture you will be able to explain why AI and ML systems introduce distinct data protection challenges beyond those addressed by conventional data security controls.
- By the end of this lecture you will be able to describe the privacy risks specific to training datasets, including the memorization problem and its implications for GDPR right-to-erasure compliance.
- By the end of this lecture you will be able to define data poisoning, model extraction, membership inference attacks, and adversarial examples, and explain how each threatens the security of an ML system.
- By the end of this lecture you will be able to apply data minimization, differential privacy, dataset validation, and data provenance documentation to a machine learning pipeline.
- By the end of this lecture you will be able to identify the security controls required at each stage of the ML pipeline, from data ingestion through model deployment and ongoing inference monitoring.

Introduction

The August 2026 CCSP exam outline adds sub-domain 2.9 — AI and ML data protection — as an explicitly testable area for the first time. This is not a theoretical addition: machine learning systems are now deployed across virtually every industry sector, processing sensitive personal data at scale, and generating decisions that affect individuals in consequential ways. Cloud security professionals who did not previously need to think about training pipelines and model APIs now need to. If you have been studying from materials predating the August 2026 outline, this lecture closes a gap that could cost you points on exam day.

The data protection challenges introduced by AI and ML systems fall into two broad categories. The first concerns the input side of the problem: the datasets used to train models. Training data is often sensitive, and the volume and richness of data required to produce capable models means that privacy risks scale with model quality. The second category concerns the deployed model itself: once trained, a model is an organizational asset that can be stolen, manipulated, or probed in ways that expose information about its training data even when that data is no longer directly accessible. Both categories require security controls, and both connect to regulatory obligations that cloud security professionals are expected to understand.

For the CCSP examination, sub-domain 2.9 tests conceptual understanding rather than implementation depth. You need to know what each threat is, why it matters from a data protection perspective, and which class of controls addresses it. This guide builds that framework systematically across the full ML pipeline.

Core Concepts Explained

The Memorization Problem and GDPR Right-to-Erasure Compliance

Definition: The memorization problem describes the phenomenon in which a trained machine learning model encodes characteristics of individual training examples in its weights and parameters, in some cases to a degree that allows those characteristics to be reconstructed or inferred from the model's outputs. This creates a compliance challenge when the underlying data is subject to deletion rights.

Business relevance: The GDPR right to erasure (Article 17) requires organizations to delete personal data upon request when no overriding legitimate purpose for retention exists. For conventional databases this means deleting a record. For ML systems it is less clear: the personal data no longer exists as a discrete record — it has been absorbed into model weights through training. Regulators in the EU and UK are actively examining whether a trained model that has memorized personal data constitutes processing of that data that must be addressed in response to an erasure request.

Cloud Security Practitioner relevance: A Cloud Security Practitioner advising on ML system design raises this issue during the privacy impact assessment stage — before training begins. The practitioner recommends data minimization as the primary mitigation: if personal data is not included in the training set, it cannot be memorized. Where personal data is necessary for training, the practitioner documents the legal basis for processing and the organization's planned response to erasure requests, which may include model retraining or machine unlearning techniques.

Real-world example: A European healthcare company built a clinical decision support model trained on ten years of patient diagnostic records. During development, the privacy team raised the memorization problem: the model might encode characteristics of individual patients whose records happened to appear infrequently in the dataset — a known risk factor for memorization. A patient subsequently submitted an erasure request. The company could delete the patient's record from its database, but the legal team concluded that the trained model might retain the patient's data in a memorized form. The company's response included committing to retrain the model with the patient's records excluded at the next scheduled model update cycle, documenting this commitment in the erasure response, and implementing differential privacy noise in the next training run to reduce future memorization risk. The response satisfied the supervisory authority's inquiry.

Risks or limitations: The risk compounds when models are trained on small datasets where individual records have disproportionate influence on model behavior — a configuration that maximizes memorization risk. The best practice is to apply differential privacy during training, which introduces mathematically calibrated noise that provides a measurable privacy guarantee for individuals in the training set, and to avoid training on datasets where any individual's data is irreplaceable to model performance.

Data Poisoning: Integrity Attacks on Training Data

Definition: Data poisoning is an integrity attack in which an adversary deliberately introduces malicious or misleading data into a training dataset, with the goal of corrupting the model's behavior — either by degrading overall performance or by causing the model to produce specific incorrect outputs in targeted scenarios.

Business relevance: Organizations that deploy ML models in security-critical decision-making roles — fraud detection, malware classification, credit risk assessment, access control — face the risk that a compromised training pipeline could produce a model that behaves correctly in most cases but fails systematically in exactly the scenarios an attacker has targeted. The failure may not be apparent during standard evaluation testing because the attacker has designed it to activate only on specific inputs.

Cloud Security Practitioner relevance: A Cloud Security Practitioner responsible for securing the ML pipeline treats dataset integrity with the same rigor applied to code integrity. This means controlling who can contribute data to training pipelines (least-privilege access to data ingestion endpoints), validating all input data before use (schema validation, anomaly detection on new batches, provenance verification), and maintaining audit logs of all contributions to the training dataset so that if a poisoning attack is detected, the malicious data can be traced to its source and removed.

Real-world example: In 2021, researchers demonstrated a practical data poisoning attack against a spam detection classifier used by a major email provider. By contributing a carefully crafted set of spam emails to a community threat feed that fed directly into the model's retraining pipeline, the attackers were able to cause the retrained model to misclassify a specific category of phishing emails as legitimate. The attack worked because the threat feed contribution process had no validation step and no source attribution logging — any contributor could add data without authentication. After the attack was detected through anomaly analysis of model output distributions, the provider implemented signed dataset contributions with contributor attribution, automated anomaly detection on new training batches, and a staging environment where retrained models are evaluated against held-out adversarial test cases before production deployment.

Risks or limitations: The subtlest poisoning attacks are designed to pass standard accuracy benchmarks while introducing targeted misbehavior. Standard evaluation testing is insufficient defense. The best practice is to include adversarial test cases specifically designed to detect targeted poisoning behavior, and to evaluate model behavior on these cases as a mandatory step before any model update is deployed to production.

Model Extraction and Intellectual Property Risk

Definition: Model extraction — also called model theft — occurs when an adversary systematically queries a model's inference API and uses the pattern of responses to reconstruct a functionally equivalent copy of the model, without ever obtaining the model weights or training data directly.

Business relevance: A trained model represents a significant organizational investment — in data acquisition, compute costs, and expertise. Model extraction allows a competitor or adversary to obtain the functional equivalent of that investment by querying the public API. Beyond intellectual property loss, a stolen model can be used to develop more effective adversarial examples against the original, and if the model encodes sensitive training data, the extracted copy may preserve those privacy risks.

Cloud Security Practitioner relevance: A Cloud Security Practitioner implementing model serving infrastructure adds rate limiting and query monitoring to inference API endpoints as standard controls. Rate limiting constrains the volume of queries any single client can make in a given period, raising the cost and time required for systematic extraction. Query monitoring uses anomaly detection to identify query patterns characteristic of extraction attempts — for example, clients who systematically probe the model with inputs from different regions of the input space — and triggers alerting and potential blocking.

Real-world example: A financial services firm deployed a proprietary credit risk model through a REST API. The model was trained on five years of the firm's own loan performance data — a dataset that could not be replicated by a competitor. A competing firm began querying the API through a network of rotating IP addresses, submitting synthetic applicant profiles designed to map the model's decision boundaries. The firm's API gateway detected the pattern through query volume analysis: the rotating IP addresses collectively exceeded the per-user rate limit by querying in coordination. The security team implemented application-layer rate limiting keyed to authenticated client IDs rather than source IPs, required API key registration with a terms-of-service acknowledgment prohibiting systematic querying for model reconstruction, and added confidence score perturbation to model outputs — a technique that introduces noise in the returned

confidence values without affecting the primary prediction, raising the number of queries required for accurate extraction by an order of magnitude.

Risks or limitations: Rate limiting keyed to source IP address is easily circumvented through IP rotation. The best practice is to authenticate API consumers and key rate limits to authenticated identities, combined with behavioral analysis that detects coordinated querying across multiple identities.

Membership Inference Attacks and Training Data Privacy

Definition: A membership inference attack is a privacy attack in which an adversary crafts queries specifically designed to determine whether a particular individual's data was included in a model's training set. The model's differential behavior on training examples versus non-training examples — a property called overfitting — enables this inference.

Business relevance: When models are trained on sensitive personal data — medical records, financial data, communications content — a successful membership inference attack is functionally equivalent to a data breach: the attacker has learned that a specific individual's sensitive data was processed by the organization, potentially exposing both the individual and the organization to regulatory and reputational consequences.

Cloud Security Practitioner relevance: A Cloud Security Practitioner advising on model deployment evaluates the sensitivity of the training data against the risk of membership inference before deciding on the model's deployment architecture. Models trained on highly sensitive data that are exposed through public APIs present a high membership inference risk. Mitigations include training with differential privacy (which limits what membership inference can reveal), restricting model access to authenticated internal consumers only, and returning only prediction classes rather than confidence scores (which are the primary signal used in membership inference attacks).

Real-world example: Researchers from a university privacy lab demonstrated a membership inference attack against a publicly accessible commercial health prediction API in 2020. By querying the model with records drawn from a known disease registry — a dataset containing real patient information — and comparing the model's confidence scores on these records against confidence scores on synthetic records with similar demographic characteristics, the researchers were able to determine with greater than 80 percent accuracy which patients from the registry had been included in the model's training set. The model provider had not considered membership inference as a deployment risk; after the disclosure, the provider implemented confidence score bucketing (returning high, medium, or low rather than the raw probability) and required authenticated access for all API calls, reducing the attack effectiveness significantly.

Risks or limitations: Returning raw confidence scores or probability values dramatically increases the effectiveness of membership inference attacks because these values directly reflect how familiar the model is with a specific input. The best practice is to return only the prediction class in consumer-facing APIs, reserve confidence scores for authenticated internal consumers with a legitimate need, and train with differential privacy for any model that will be deployed with even restricted external access.

Adversarial Examples and the Integrity of Model Decisions

Definition: Adversarial examples are specially crafted inputs that cause a machine learning model to produce incorrect outputs with high confidence. These inputs are designed to appear legitimate — sometimes indistinguishable from valid inputs to a human observer — while systematically exploiting the model's learned decision boundaries.

Business relevance: In security-critical applications such as malware classification, fraud detection, network intrusion detection, and access control systems, adversarial examples can be used to systematically evade

detection. An adversary who understands the model's decision boundaries can craft inputs that pass detection with high confidence, undermining the security function the model provides.

Cloud Security Practitioner relevance: A Cloud Security Practitioner incorporating ML models into security workflows includes adversarial robustness evaluation in the model acceptance testing process. This means testing candidate models against known adversarial example generation techniques — such as Fast Gradient Sign Method and Projected Gradient Descent — before deployment, and establishing a monitoring process that flags anomalous input patterns that might indicate adversarial probing of the deployed model.

Real-world example: A cybersecurity company deployed an ML-based malware classifier as part of its endpoint protection product. Independent security researchers demonstrated that by appending a small number of bytes to a malware binary — bytes chosen through adversarial optimization to shift the model's classification toward benign — the model could be caused to classify known malware as safe with high confidence. The modification did not affect the malware's executable behavior; the payload functioned identically. The company's model development team responded by implementing adversarial training — incorporating adversarially crafted examples into the training dataset to harden the model's decision boundaries — and adding an ensemble of classifiers with different architectural approaches, because adversarial examples designed to evade one model architecture are less likely to evade a structurally different model covering the same classification task.

Risks or limitations: Models deployed in security-critical roles without adversarial robustness testing are vulnerable to systematic evasion. The best practice is to treat adversarial example evaluation as a non-negotiable part of the acceptance testing process for any model used in a security decision-making role, and to retrain with adversarial examples when new attack techniques are identified.

Practical Applications

The following scenarios illustrate how Cloud Security Practitioners apply AI and ML data protection principles across realistic organizational contexts.

Building a Privacy-Preserving ML Pipeline for a Regulated Dataset

A Cloud Security Practitioner at a large insurance company is asked to advise on the data protection architecture for a new ML model that will use policyholder claims data to predict fraud risk. The training dataset contains names, dates of birth, medical procedure codes, and claims amounts — all personal data under applicable privacy regulations. The practitioner begins by leading a data minimization review: working with the data science team to determine the minimum set of features required to achieve the target model accuracy, and replacing directly identifying fields with pseudonymized identifiers where the pseudonymized version is sufficient for training purposes. For fields that cannot be removed without degrading model performance, the practitioner recommends training with differential privacy using a validated library that provides formal mathematical privacy guarantees for individuals in the dataset. The practitioner then establishes a data provenance log that records the legal basis for using each data source, the data processing agreement with each source system, the transformations applied before training, and who had access to the dataset during preparation. This documentation satisfies both the regulatory compliance requirement and the audit trail requirement in the event the model's training data is questioned in a future claim dispute.

Implementing Inference API Security for a Deployed Model

A Cloud Security Practitioner at a healthcare analytics company is responsible for securing the inference API for a clinical risk scoring model deployed as an internal service. The model was trained on patient records and

produces a risk score used by clinical staff to prioritize patient outreach. Because the training data is highly sensitive, the practitioner evaluates the membership inference risk profile: the model returns a continuous risk score between 0 and 1, which is a high-information signal for membership inference. The practitioner implements three mitigations. First, API access is restricted to authenticated internal consumers using short-lived bearer tokens — no public access is permitted. Second, the response schema is modified to return a bucketed risk level (low, medium, high, critical) rather than the raw score, eliminating the high-precision signal required for effective membership inference. Third, the API gateway implements per-client rate limiting with behavioral anomaly detection that triggers a security alert if any authenticated client queries the model more than 500 times per hour — a threshold above normal clinical use patterns but below the volume required for systematic extraction or inference attacks. These controls reduce the membership inference and model extraction risk without affecting the clinical workflow.

Responding to a Data Poisoning Incident in a Retraining Pipeline

A Cloud Security Practitioner at a cybersecurity vendor is investigating an anomaly in the company's network intrusion detection model. Monitoring has flagged that the model's false negative rate on a specific category of lateral movement traffic has increased significantly following the most recent retraining cycle. The practitioner begins by reviewing the audit log for the retraining dataset to identify what new data was added before the anomalous cycle. The log reveals that a threat intelligence feed — recently integrated as an automated training data source — contributed several thousand labeled examples in the weeks before retraining. The practitioner pulls a sample of the contributed examples and identifies that a subset of traffic samples labeled as benign show characteristics consistent with the lateral movement category the model now misclassifies. The practitioner implements an immediate rollback to the previous model version while the investigation continues. The poisoned examples are traced to a specific feed contributor using the provenance log; the contributor's access is suspended and the relevant samples are removed. The practitioner then works with the engineering team to implement validation gates on all new training data contributions: schema validation, statistical distribution checks to detect batch-level anomalies, and human review for any contribution from a new or recently added source before it is admitted to the training pipeline.

Best Practices

- **Apply data minimization to training datasets before the training process begins.** Audit the proposed training dataset against the model's intended function and remove or pseudonymize any data elements not required for achieving the target accuracy. Data that is never included in the training set cannot be memorized, cannot be exposed through membership inference, and does not create erasure compliance obligations. Document the minimization decisions and the business rationale for each retained element.
- **Train models that will be externally accessible or trained on sensitive personal data using differential privacy.** Differential privacy introduces mathematically calibrated noise during training that provides a formal, measurable privacy guarantee for individuals in the dataset. The guarantee can be quantified using the epsilon parameter — lower epsilon values provide stronger privacy protection at some cost to model accuracy. Work with the data science team to select an epsilon value appropriate to the sensitivity of the training data and the intended deployment context.
- **Implement dataset provenance logging covering the legal basis, source, transformation steps, and access history for all training data.** Provenance documentation enables regulatory compliance (demonstrating the legal basis for processing personal data in training), supports incident investigation (tracing the source of poisoned examples), and satisfies audit requirements. The provenance log should be maintained in an

access-controlled, immutable record separate from the training pipeline itself, and should be updated at each stage where the dataset changes.

- **Restrict model inference API access to authenticated consumers and return only the minimum output required for the intended use case.** Anonymous or unauthenticated access to inference APIs dramatically increases exposure to model extraction and membership inference attacks. Require API key or token authentication for all consumers, key rate limits to authenticated identities rather than source IPs, and return only prediction classes rather than raw confidence scores in consumer-facing APIs. Reserve high-precision outputs for internal authenticated systems with a documented legitimate need.
- **Include adversarial robustness evaluation and data poisoning detection as mandatory gates in the model deployment pipeline.** Standard accuracy evaluation on held-out test sets does not detect targeted poisoning or adversarial vulnerabilities. Add adversarially crafted test cases to the evaluation suite, run statistical distribution checks on new training data batches before admission, and require a security review of any significant change to training data sources or collection pipelines. Models that fail adversarial evaluation should not be deployed to production roles regardless of standard accuracy metrics.

Common Mistakes and Misconceptions

Misconception: Assuming that deleting a person's records from the source database satisfies an ML model erasure obligation under GDPR.

Correct approach: Deletion of source records does not address data that has been incorporated into model weights through training. The correct approach is to assess whether the model has memorized the individual's data, document the organization's planned response in the erasure notice, and include model retraining with the records excluded in the next scheduled cycle, with differential privacy applied to reduce future memorization risk.

Misconception: Treating training data pipelines as trusted internal systems that do not require the same access controls as production data environments.

Correct approach: Training pipelines are high-value targets: compromising a training pipeline allows an attacker to poison the model without ever touching the production inference system. Apply least-privilege access controls to all components of the training pipeline — data ingestion, preprocessing, training compute, and model artifact storage — and log all access with the same rigor as production data environments.

Misconception: Relying on IP-based rate limiting to prevent model extraction attacks.

Correct approach: IP-based rate limiting is easily circumvented through IP rotation and distributed querying. Key rate limits to authenticated client identities, implement behavioral anomaly detection that identifies coordinated querying patterns across multiple identities, and add output perturbation techniques such as confidence score bucketing to raise the cost of systematic extraction.

Misconception: Returning raw confidence scores or probability values from inference APIs exposed to external consumers.

Correct approach: Raw confidence scores provide the primary signal used in membership inference attacks and also enable more effective model extraction. Return only bucketed risk levels or prediction classes in consumer-facing APIs. If continuous scores are required for a specific internal use case, restrict access to authenticated internal systems and document the business justification.

table: ML Pipeline Security Controls by Stage

Apply the following security controls at each stage of the machine learning pipeline. Each stage represents a distinct attack surface; gaps at any stage can undermine the security of the deployed system.

Pipeline Stage	Primary Threats	Required Controls	Best-Practice Indicators
Data Ingestion	Data poisoning, unauthorized data inclusion	Source authentication and attribution; schema and distribution validation; access logging	All data contributions are signed and attributed; anomaly detection on new batches
Data Preparation	Re-identification, privacy exposure	Data minimization review; pseudonymization or anonymization; provenance logging with legal basis	Minimization decisions documented; no unnecessary PII retained
Model Training	Memorization, training infrastructure compromise	Differential privacy where sensitive data is used; least-privilege access to compute; artifact encryption	DP epsilon documented; training compute access logged; model artifacts encrypted at rest
Model Validation	Adversarial vulnerabilities, targeted poisoning	Adversarial test suite evaluation; performance comparison against prior version; anomaly review	Adversarial test cases in evaluation suite; poisoning detection gate passed
Model Deployment	Model extraction, membership inference	Authenticated API access; rate limiting keyed to identity; output bucketing or perturbation	No anonymous API access; rate limits active; raw scores not exposed externally
Inference Monitoring	Systematic extraction, adversarial probing	Query volume monitoring; behavioral anomaly detection; incident response plan for model compromise	Alerting on high-volume querying; model compromise included in IR runbooks

How to use this: Review this table when onboarding any new ML system into your cloud environment. Security requirements should be confirmed before the training pipeline is built, not after the model is deployed.

Knowledge Check

Test your understanding before moving on. Answers are shown below each question.

Q1. A trained ML model was built using patient health records. A patient submits a GDPR erasure request. The organization has deleted the patient's records from its source database. What additional obligation may remain with respect to the trained model?

- A) A. No additional obligation exists because the training records have been deleted from the source system.
- B) B. The organization may need to assess whether the model memorized the patient's data, document its response, and plan for model retraining with the records excluded to address the potential residual personal data in the model weights.
- C) C. The organization must immediately destroy the trained model and retrain from scratch.
- D) D. GDPR erasure rights do not apply to data processed through machine learning systems.

Answer: B. The organization may need to assess whether the model memorized the patient's data, document its response, and plan for model retraining with the records excluded to address the potential residual personal data in the model weights.

Q2. An adversary introduces carefully crafted examples into a threat intelligence feed that feeds automatically into a malware classifier's retraining pipeline. The crafted examples cause the retrained model to misclassify a specific malware variant as benign. Which threat does this scenario illustrate?

- A) A. Membership inference attack
- B) B. Adversarial example attack
- C) C. Data poisoning attack
- D) D. Model extraction attack

Answer: C. Data poisoning attack

Q3. A company exposes an ML model through a public REST API that returns a continuous probability score between 0 and 1 alongside each prediction. Which security risk does this configuration most directly increase?

- A) A. Data poisoning, because the continuous scores reveal which training examples are most influential.
- B) B. Membership inference attacks, because high-precision confidence scores allow adversaries to determine whether specific individuals were included in the training set.
- C) C. Adversarial examples, because continuous scores make it easier to craft inputs that cross decision boundaries.
- D) D. Model extraction, because the API itself exposes the model's weights directly.

Answer: B. Membership inference attacks, because high-precision confidence scores allow adversaries to determine whether specific individuals were included in the training set.

Q4. A Cloud Security Practitioner is designing the inference API for an internal ML model trained on sensitive financial data. Which combination of controls best addresses the membership inference and model extraction risks?

- A) A. Require API key authentication, return only prediction categories rather than raw confidence scores, and implement per-client rate limiting with behavioral anomaly detection.
- B) B. Deploy the API behind a web application firewall configured to block known malicious IP addresses.
- C) C. Encrypt the model weights at rest and in transit to prevent them from being obtained by an attacker.
- D) D. Limit the model to internal use and require VPN access for all consumers.

Answer: A. Require API key authentication, return only prediction categories rather than raw confidence scores, and implement per-client rate limiting with behavioral anomaly detection.

Q5. Which technique provides a mathematically measurable privacy guarantee for individuals in a training dataset, limiting what membership inference attacks can reveal about whether a specific individual's data was used in training?

- A) A. Pseudonymization
- B) B. Data masking
- C) C. Differential privacy
- D) D. Tokenization

Answer: C. Differential privacy

Reflection Questions

1. Consider an ML system currently operating in your organization or a case study you have studied. Which stages of the ML pipeline from data ingestion through inference monitoring had documented security controls, and which stages were implicitly trusted as internal systems?
2. The GDPR right to erasure was written before large-scale machine learning was common practice. How does the memorization problem change your understanding of what it means to have deleted personal data?
3. If a model extraction attack is discovered against a publicly accessible API, what immediate and longer-term responses would you recommend? Which controls would have been most effective in preventing the attack?
4. Differential privacy trades some model accuracy for privacy guarantees. How would you make the case to a data science team that this trade-off is worth accepting for models trained on sensitive personal data?
5. Consider the security controls in the ML Pipeline Security Controls table above. For an ML system you are familiar with, which row represents the greatest current gap between current practice and the recommended controls?

Key Takeaways

- Sub-domain 2.9 is new to the August 2026 CCSP exam outline and explicitly testable — candidates using older study materials have a gap in this area.
- The memorization problem means that personal data can become encoded in trained model weights, creating GDPR erasure compliance obligations that extend beyond deleting source records.
- Data poisoning is an integrity attack targeting the training pipeline; dataset validation, provenance logging, and least-privilege access to ingestion endpoints are the primary preventive controls.
- Model extraction allows an adversary to reconstruct a model's functional behavior through systematic API querying, without ever accessing model weights directly; authenticated access and per-client rate limiting are the primary mitigations.
- Membership inference attacks exploit high-precision confidence scores to determine whether specific individuals were included in training data; returning only bucketed prediction categories significantly reduces this risk.
- Adversarial examples are specially crafted inputs that cause models to produce incorrect outputs with high confidence; adversarial robustness evaluation should be a mandatory gate in the deployment pipeline for security-critical model applications.
- The standard data security controls — encryption, access control, audit logging, rate limiting, incident response planning — apply to ML pipelines, but must be extended to cover the unique attack surfaces each pipeline stage presents.
- For the CCSP examination, the key concepts from sub-domain 2.9 are: memorization problem, data poisoning, model extraction, membership inference, adversarial examples, and the application of standard data security controls across the full ML pipeline.

Recommended Next Actions

Put this lecture to work within the next seven days:

6. Identify any ML systems currently deployed or in development in your organization and map each one to the ML Pipeline Security Controls table in this guide, noting which controls are in place and which represent gaps.

7. Review the GDPR right-to-erasure response process for your organization and determine whether it addresses ML models that may have been trained on personal data; if not, document a remediation approach.
8. Locate the training data provenance documentation for any ML system you are responsible for and verify it includes the legal basis for processing, data source records, transformation logs, and access history.
9. Add adversarial example test cases to the evaluation suite of any ML model deployed in a security decision-making role, and confirm the acceptance testing process includes a mandatory adversarial robustness gate.